

DESIGNING DATA-INTENSIVE APPLICATIONS · VISUAL COMPANION

Chapter 10

Batch Processing

Bounded inputs, derived outputs, and the shuffle that brings related data together

Source: [claude-skills / ch10-batch-processing.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part III: Derived Data

Table of Contents

1. Core Idea
2. Framework: MapReduce + HDFS
3. Framework: Dataflow Engines
4. Unix Philosophy & Uniform Interface
5. The Shuffle
6. Join Algorithms
7. Skew Handling
8. Derived Data & Batch Output
9. MapReduce vs MPP & Graph Processing
10. Declarative Optimization
11. Mental Models
12. Anti-Patterns
13. Worked Example: Log Analysis Three Ways
14. Problems Each Mechanism Solves
15. Connects To (Cross-Chapter)
16. Key Takeaways
17. Junior SWE Checkpoints

1. Core Idea

Batch jobs read a **bounded, immutable input**, produce a **completely new output**, and have **no other side effects**. This one discipline — derived output, immutable input — buys you:



Fault tolerance

Retry any task safely — partial output is discarded.



Human fault tolerance

Bad code? Re-run against the same input.



Composability

Any job's output can feed any future job.

MapReduce is "Unix distributed": HDFS is the filesystem, a job is a process, and the **shuffle** is the always-on **sort** between map and reduce. Everything else in the chapter is join algorithms and the engines that made the sort/materialization optional.

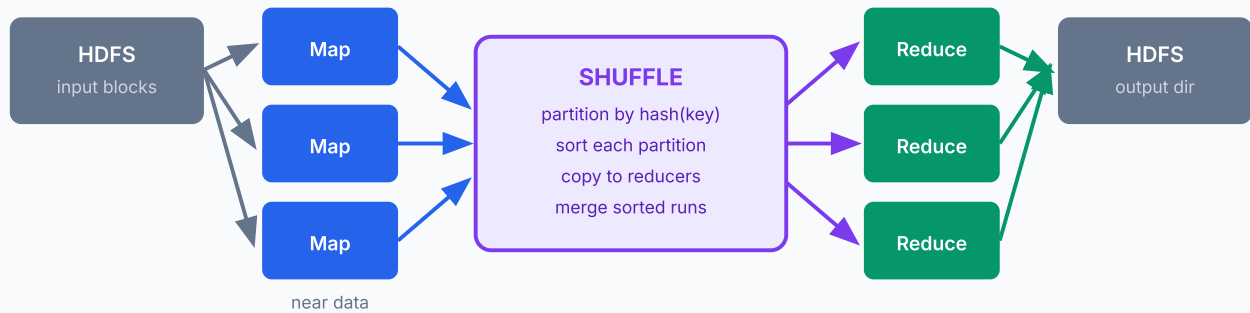
Immutable inputs + side-effect-free tasks give exactly-once *output* semantics: the final result equals a failure-free run even though tasks were retried. Stronger than anything an online service gets.

2. Framework: MapReduce + HDFS (Hadoop)

Also: Hadoop MapReduce, Pig, Hive on MR. **Original big-data batch default** — arbitrary code over petabyte datasets.

When	Huge bounded datasets, arbitrary code (ML feature engineering, index building, ETL into a warehouse), multitenant clusters where tasks get preempted, jobs long enough that at least one task will fail.
How	Input directory in HDFS split by file block; one map task per block, scheduled near the data ("putting the computation near the data"). Mapper extracts key-value pairs; framework partitions by hash of key, sorts each partition to local disk (SSTable-style), reducers fetch and merge — the shuffle . Reducer sees each key with an iterator over all its values, writes output to HDFS. Chained jobs = workflows, wired by directory names, orchestrated by schedulers (Airflow, Oozie, Luigi).
Why it works	Stateless callbacks + immutable inputs mean the framework can retry any failed task and discard partial output; job output is all-or-nothing. Designed for Google's mixed-priority datacenters where a 1-hour task had ~5% chance of preemption — aggressive disk materialization is a bet that failure is frequent.
Failure mode	Fully materializing intermediate state to HDFS between every job. Downstream jobs wait for every straggler in the upstream job; redundant mapper stages just re-read what a reducer wrote. Slow in the failure-free case.

MapReduce job — map → shuffle → reduce



Chained jobs = workflow via HDFS directory names

Job1/output/ → Job2/input/ · orchestrated by Airflow / Oozie / Luigi

Figure 2.1 — MapReduce: stateless map/reduce callbacks, shuffle co-locates keys, output written to HDFS.

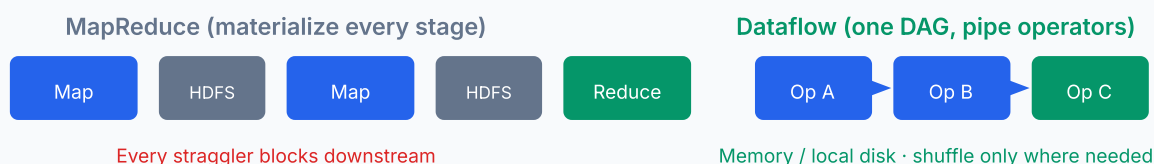
Real systems: Hadoop MapReduce, Pig, Hive (MR backend), Google's original index builder, early Facebook data warehouse ETL.

3. Framework: Dataflow Engines (Spark, Flink, Tez)

Also: Spark, Flink, Tez on YARN. **Modern multi-stage batch default** — one DAG instead of chained MR jobs.

When	Multi-stage workflows where intermediate state is throwaway; iterative/interactive work; anywhere MapReduce would chain 50+ jobs (recommendation pipelines).
How	Whole workflow is one job, a DAG of operators (generalized map/reduce). Sorting only where needed; partitioned-without-sort and broadcast connections available; intermediate state in memory or local disk, streamed operator-to-operator pipe-style. Tez is a thin library on YARN's shuffle service; Spark and Flink are full frameworks.
Why it works	Avoids materializing every stage to HDFS — faster when failures are rare. Operators piped directly; shuffle only on demand.
Failure mode	Fault tolerance via recomputation instead of durability — Spark tracks lineage with RDDs, Flink checkpoints operator state. Only works if operators are deterministic ; hash-iteration order, random numbers, or clock reads silently break recovery and force killing downstream operators too. If intermediate data is small or compute is expensive, materializing beats recomputing.

MapReduce vs Dataflow engine



Recovery: Spark RDD lineage · Flink checkpoints · requires deterministic operators
Non-determinism (RNG, clock, hash-order iteration) → corrupt recovery → cascade restart

Figure 3.1 — Dataflow engines treat the workflow as one DAG; MR materializes every intermediate to HDFS.

Determinism is the precondition for cheap recomputation. Materialize when tasks die often (preemption-heavy clusters); recompute when they don't.

4. Unix Philosophy & Uniform Interface

Unix principles

Do one thing well; expect your output to be another program's input; immutable inputs; separate logic from wiring (stdin/stdout, pipes).

`sort` spills to disk and parallelizes — the pipeline scales past memory.

Uniform interface

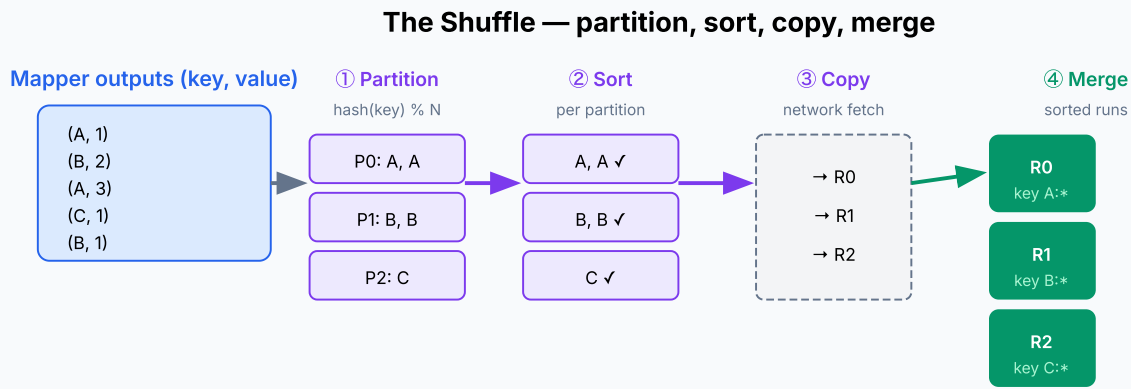
In Unix: a file (byte stream, usually newline-separated ASCII). In Hadoop: HDFS files (Avro/Parquet replace fragile text parsing — Ch 4).

```
cat access.log | awk '{print $7}' | sort | uniq -c | sort -rn | head -5
# awk = map · sort = shuffle · uniq -c = reduce · scales past RAM via sort spill-mer
```

Sorting vs hashing is the working-set question. Fits in memory → hash table (Ruby script, broadcast join). Doesn't → sort-merge with sequential disk I/O (Unix `sort`, the shuffle). Same trade-off at every scale.

5. The Shuffle

Partition mapper output by hash of key, sort, copy to reducers, merge. Deterministic, no randomness despite the name. It is what **"brings related data to the same place"** — the mapper's key is a destination address.



Keys are addresses

Emitting (key, value) = sending a message; shuffle = delivery network
Joins, GROUP BY, sessionization, Pregel — all the same move: bring related data to one place
Sort-spill-merge = SSTable/LSM machinery (Ch 3) · sequential I/O beats random access

Figure 5.1 — Shuffle: partition by $\text{hash}(\text{key})$, sort locally, copy to reducers, merge sorted runs (SSTable-style).

6. Join Algorithms

Three join strategies — know them cold and the input properties that select each. Physical layout (partition count, key, sort order in Hive metastore/HCatalog) determines which join is even legal.

Join type	When	How	Cost
Reduce-side sort-merge	No assumptions about input; default fallback	Mappers over both inputs emit join key; shuffle co-locates them; secondary sort puts dimension record (e.g. user profile) first so reducer holds one record in memory while streaming facts	Full shuffle of both sides
Broadcast hash join	Small side fits in memory	Map-only job; each mapper loads entire small input into in-memory hash table, streams large input (Hive "MapJoin", Pig "replicated join")	No shuffle at all
Partitioned hash join	Both inputs pre-partitioned identically (same key, hash, partition count)	Each mapper hashes only its partition (Hive "bucketed map join")	No shuffle; requires prior co-partitioning
Map-side merge join	Both inputs partitioned AND sorted on join key	Stream-merge without memory limits — like Unix <code>join</code> on sorted files	No shuffle; requires sorted co-partitioned inputs

Three join strategies — decision tree

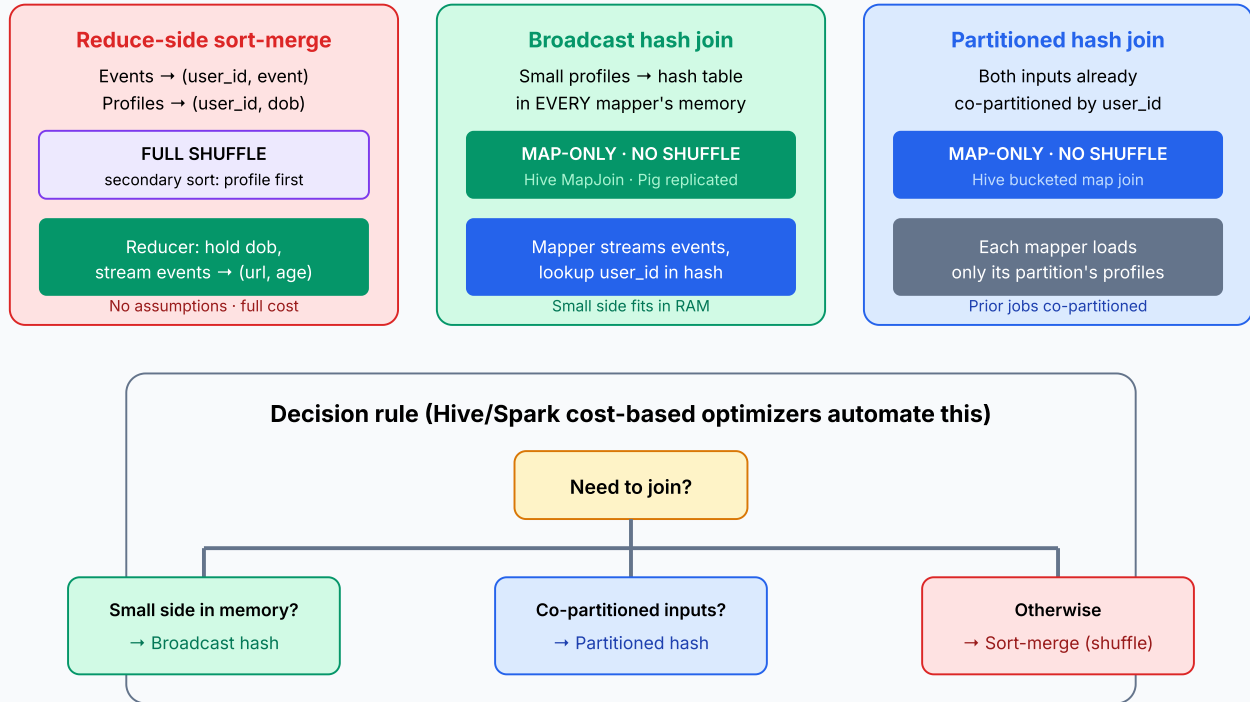


Figure 6.1 — Join selection: broadcast (small side), partitioned hash (co-partitioned), sort-merge (fallback).

Physical layout is metadata that matters: partition count, key, and sort order of a dataset determine which join is even legal. Hive metastore / HCatalog track this.

7. Skew Handling

Linchpin objects (celebrities) overload one reducer — the job is only as fast as its slowest reducer. Hash partitioning sends all records for a hot key to the same reducer.

Symptom

99 reducers finish in 2 minutes; one reducer runs 3 hours processing a celebrity's 500M fan events.

Skew join fix

Send the hot key's records to **random reducers** and replicate the other join side to all of them (Pig/Hive *skew join*).

GROUP BY fix

Two-stage aggregation: first stage adds random salt to hot keys and partial-aggregates; second stage removes salt and final-aggregates.

Skew: one hot key overloads a reducer

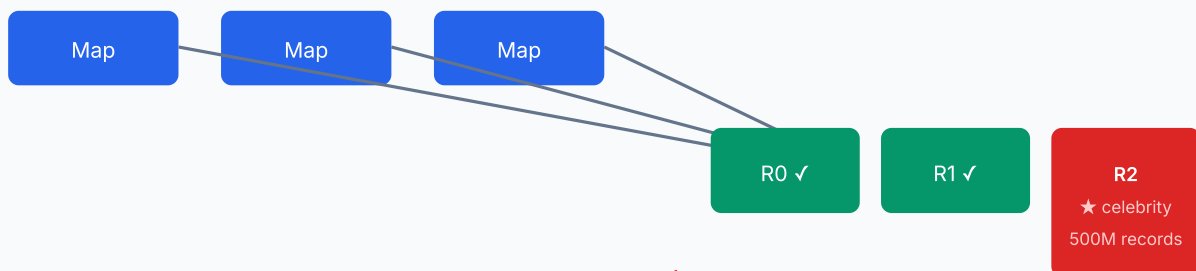


Figure 7.1 — Hash partitioning sends all celebrity records to one reducer.

8. Derived Data & Batch Output

Batch output = **derived data**, rebuilt wholesale from systems of record. If it's wrong, fix the code and re-run.

Pattern	How	Examples
Immutable index files	Documents in → search index files out; swap atomically on serve	Google's original use case, autocomplete trie rebuild
Read-only KV store inside job	Build store inside the job, bulk-load to serving layer	Voldemort, Terrapin, ElephantDB, HBase bulk load
Database inside the job, bulk-swap on serve	Rollback = serve previous artifact; no partial state visible	Voldemort pattern — trivial rollback

Never write record-at-a-time to an external DB from a job. It's slow, hammers the DB, and leaks side effects that break the all-or-nothing guarantee. Build derived artifacts offline; bulk-swap on serve.

Batch input is bounded, so jobs finish. Remove that assumption and you get stream processing (Ch 11).

9. MapReduce vs MPP & Graph Processing

MapReduce vs MPP databases

Dimension	MapReduce / Hadoop	MPP databases
Architecture	Raw byte storage ("data lake", sushi principle: raw data is better, schema-on-read)	Monolithic, schema-first, SQL
Execution	Arbitrary code, task-level retry, disk-eager	In-memory, abort-and-retry whole queries
Convergence	Hive/Spark grew cost-based optimizers, columnar reads, vectorized execution — they're meeting in the middle	

Pregel / BSP for graphs

Vertex holds state across iterations, sends messages along edges, delivered in the next superstep; checkpointed for fault tolerance. But if the graph fits on one machine, **single-node processing usually wins** — cross-machine message traffic dominates.

Iterative graph algorithms as chained MapReduce jobs — every iteration rereads and rewrites the entire dataset even when almost nothing changed. Use Pregel/GraphX/Giraph for large graphs; don't distribute a graph job that fits on one machine.

10. Declarative Optimization

Declarative creep: specify joins relationally and the optimizer picks the join algorithm; keep callback UDFs for the code SQL can't express. Best of both worlds.

Hive, Spark SQL, Flink SQL cost-based optimizers automate the broadcast / partitioned hash / sort-merge decision based on table statistics, partition metadata, and memory estimates.

11. Mental Models

1. Hadoop = distributed Unix

HDFS is the filesystem, a MapReduce job is a process that always pipes through `sort`, workflows are pipelines that write every stage to a temp file. Dataflow engines put the actual pipes back.

2. Keys are addresses

Emitting (key, value) is sending a message; the shuffle is the delivery network. Joins, GROUP BY, sessionization, and Pregel are all the same move: bring related data to one place, then run simple single-threaded logic.

3. Fault tolerance is a bet on failure rate

Materialize when tasks die often (preemption-heavy clusters); recompute when they don't. Determinism is the precondition for cheap recomputation.

4. Sorting vs hashing

Fits in memory → hash table (Ruby script, broadcast join). Doesn't → sort-merge with sequential disk I/O (Unix sort, the shuffle). Same trade-off at every scale.

12. Anti-Patterns

Anti-pattern	Why it fails
Querying a remote DB per record inside a job	Network round-trips kill throughput and make the job non-deterministic. Snapshot the DB into the distributed filesystem instead.
Writing job output directly into a live database	Overwhelms it; partial failures leave visible half-written state; breaks all-or-nothing guarantee.
Ignoring skew	One celebrity key makes the whole workflow wait on one reducer.
Non-deterministic operators under recomputation recovery	Hash-order iteration, unseeded RNG, clock reads corrupt downstream state; forces cascading restarts.
Iterative graph algorithms as chained MapReduce jobs	Every iteration rereads and rewrites the entire dataset even when almost nothing changed.
Distributing a graph job that fits on one machine	Cross-machine message traffic dominates; single-node wins.

13. Worked Example: Log Analysis Three Ways

Unix — top-5 URLs from nginx log

```
cat access.log | awk '{print $7}' | sort | uniq -c | sort -rn | head -5
```

`awk` extracts the key (map), `sort` brings equal URLs adjacent (shuffle), `uniq -c` aggregates adjacent runs (reduce). Scales past RAM because `sort` spills sorted segments to disk and merges them — the SSTable trick. The in-memory alternative (hash-table counter script) wins only while distinct URLs fit in memory.

MapReduce — same job, one machine → thousands

```
Mapper: emit (url, 1) for each request
Framework: shuffle (partition + sort by url)
Reducer: count values per url → (url, count)
"Top 5" needs a SECOND job – another sort stage = another MapReduce job chained via
```

Now join user profiles — which age groups view which pages

Activity events carry only `user_id` ; profiles live in a user database snapshot.

- 1 Default: reduce-side sort-merge join.** One mapper set over events, one over profile snapshot, both keyed by `user_id` ; secondary sort delivers profile record first; reducer holds one date-of-birth and streams events, emitting (url, viewer-age). Works on any input, costs full shuffle of both sides.
- 2 Profile table fits in memory → broadcast hash join.** Map-only job, each mapper loads all profiles into hash table, streams events. No shuffle at all.
- 3 Both inputs already partitioned by user_id → partitioned hash join.** Each mapper loads only its partition's profiles. No shuffle.

Decision rule: small side fits in memory → broadcast; co-partitioned inputs → partitioned hash; otherwise → sort-merge. Hive/Spark cost-based optimizers automate this.

14. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Hash-dedup pipeline over huge derived corpus	"URL seen? / content seen?" at 1B pages/month, 30PB corpus — can't check online per-request	Web crawler
DAG scheduler + retry-from-persisted-intermediate (MapReduce failure model)	Long multi-stage pipeline (transcoding) must resume from where it failed, not restart from scratch	YouTube-style video transcoding
Batch-rebuilt immutable read-only index artifact, swapped in atomically	Real-time updates to serving structure infeasible at this write volume; rebuild wholesale instead	Search autocomplete (weekly trie rebuild)
Offline pipeline → immutable multi-resolution derived artifact	TB-scale unstructured input must become queryable structure without live-updating in place	Google Maps (routing tiles baked offline, served from S3)
Backfill / reconciliation batch job (Lambda vs Kappa)	Stream processing alone can't cheaply fix historical mistakes or handle very late data	Ad click event aggregation
Nightly settlement-file reconciliation as derived-data auditing	Verify real-time system's ledger against independent batch-computed source of truth	Payment system (secondary)

15. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 3 Storage	The shuffle's sort-spill-merge is SSTable/LSM machinery; sequential I/O beats random access at every layer.	"Shuffle = distributed sort = same trick as LSM segments."
Ch 4 Encoding	Avro/Parquet as the typed replacement for Unix's untyped text interface.	"Schema-on-read needs typed columnar formats, not fragile CSV."
Ch 6 Partitioning	Hash-of-key partitioning and hot-spot skew reappear verbatim in the shuffle.	"Celebrity key in shuffle = same hot partition problem."
Ch 11 Stream Processing	Same computations over unbounded input; jobs never complete; Pregel's message-passing and incremental index updates foreshadow it.	"Batch = bounded input that finishes; stream = same ops, never done."
Ch 12 Future	Derived data vs systems of record becomes the organizing principle for whole architectures.	"Batch builds derived views; the log is the system of record."

16. Key Takeaways

1. The two problems every distributed batch framework solves are **partitioning** (bring related data together) and **fault tolerance** (retry safely) — everything else is optimization.
2. Immutable inputs + side-effect-free tasks give exactly-once *output* semantics: the final result equals a failure-free run even though tasks were retried. Stronger than anything an online service gets.
3. Know the three joins cold — sort-merge (no assumptions, full shuffle), broadcast hash (small side in memory), partitioned hash (co-partitioned inputs) — and the input properties that select each.
4. Physical layout is metadata that matters: partition count, key, and sort order of a dataset (Hive metastore/HCatalog) determine which join is even legal.
5. MapReduce materializes all intermediate state to HDFS; Spark/Flink/Tez treat the workflow as one DAG, keep intermediates local, and recover by recomputation — faster when failures are rare, dependent on determinism.
6. Batch output is derived data, rebuilt wholesale from systems of record. If it's wrong, fix the code and re-run — the "database inside the job, bulk-swap on serve" pattern (Voldemort) makes rollback trivial.
7. Batch input is *bounded*, so jobs finish. Remove that assumption and you get stream processing.

17. Junior SWE Checkpoints

Q1: What two problems does every batch framework solve?

Partitioning (bring related data together via shuffle) and **fault tolerance** (retry tasks safely). Everything else — join choice, engine selection — is optimization on top.

Q2: Why can batch jobs guarantee exactly-once output but online services can't?

Immutable input + side-effect-free tasks + derived output. Failed tasks are retried and partial output discarded. Final result equals a failure-free run. Online services have mutable state and side effects per request.

Q3: When do you pick broadcast vs partitioned hash vs sort-merge join?

Broadcast: small side fits in memory (map-only, no shuffle). **Partitioned hash:** both inputs co-partitioned on join key (map-only). **Sort-merge:** fallback — no assumptions, full shuffle of both sides.

Q4: Why does MapReduce materialize to HDFS between every stage?

Designed for preemption-heavy clusters (~5% chance of 1-hour task failure). Disk materialization lets downstream resume from persisted intermediate. Slow when failures are rare — hence Spark/Flink recomputation model.

Q5: What breaks Spark/Flink recomputation-based recovery?

Non-deterministic operators: hash-iteration order, unseeded RNG, clock reads. Recovery replays with different results → corrupt downstream state → cascade restart of operators.

Q6: Why never write batch output record-at-a-time to a live DB?

Slow (network per record), hammers the DB, and partial failures leave visible half-written state — breaks the all-or-nothing guarantee. Build derived artifacts offline; bulk-swap on serve.

Q7: How does batch connect to stream processing (Ch 11)?

Same computations, but batch input is bounded so jobs finish. Stream = unbounded input, jobs never complete. Batch backfill reconciles stream mistakes and handles very late data (Lambda architecture).

Glossary

Term	Definition
Shuffle	Partition by hash(key), sort, copy to reducers, merge — brings related data together
Derived data	Output rebuilt wholesale from systems of record; fix code and re-run if wrong
Broadcast hash join	Small table loaded into every mapper's memory; map-only, no shuffle
Partitioned hash join	Both inputs co-partitioned; each mapper loads only its partition
Sort-merge join	Reduce-side join via shuffle + secondary sort; no input assumptions
Skew join	Hot key sent to random reducers; other join side replicated to all
Lineage (Spark)	RDD dependency graph enabling recomputation on failure
Checkpoint (Flink)	Periodic operator state snapshot for recovery
Schema-on-read	Raw data in data lake; apply schema at query time (Hadoop sushi principle)
Pregel / BSP	Graph processing: vertex state + message passing in supersteps
Straggler	Slowest task in a stage; entire job waits for it
Secondary sort	Sort values within a key group so dimension record arrives before facts